

BiteWise

Smart AI Food Ordering & Nutrition Platform

Author: Samadarsh

Architecture Version: 1.4 (Production Grade)

Date: July 2026

Ecosystem: FastAPI, Next.js, SQLite/PostgreSQL, Streamlit, Swiggy Builders Club MCP

Repository: github.com/samadarsh/BiteWise

Live App: bite-wise-theta.vercel.app

Executive Abstract: BiteWise is a state-of-the-art intelligent application that bridges personalized nutrition coaching, household pantry management, and automated grocery provisioning with food delivery. By integrating directly into Swiggy's Model Context Protocol (MCP) staging environment, BiteWise enables an AI Agent to dynamically search restaurant menus, construct nutritionally balanced carts, and safely manage orders while enforcing strict consumer financial guardrails, multi-user isolation, and distributed resilience checks.

1. System Architecture & Database Design

BiteWise uses a decoupled microservices architecture designed to support dynamic AI interaction, low-latency client serving, and highly-secure third-party OAuth transactions. The platform separates developer testing labs from the user-facing web app to ensure stability.

Component	Technology	Description
Frontend App	Next.js 16 (React 19, TS, Tailwind v4)	A highly responsive client interface with modular components, server-side rendering, and native API integration.
Backend API	FastAPI (Python 3.14, Uvicorn, SQLAlchemy)	An asynchronous REST API layer handling core operations, user identity, Swiggy OAuth handshakes, state machines, and LLM orchestration.
Database	SQLite (Local Dev) / PostgreSQL (Production)	Database layer managed via SQLAlchemy ORM. Migrations and updates are fully controlled in production using Alembic schemas.
Dev Sandbox	Streamlit (MCP Lab Dashboard)	A single-user simulation environment utilized to test MCP tool definitions, view telemetry, and dry-run agent workflows.

Database & Migration Integrity Strategy

To achieve transition-readiness, BiteWise uses SQLAlchemy to abstract model schemas. Local execution runs against an embedded SQLite database (**nutriorder.db**). For multi-user hosting, database configurations dynamically scale to PostgreSQL. Because production systems require zero-downtime, schema changes are strictly version-controlled using Alembic migrations, fully deprecating unsafe on-startup database mutations like *Base.metadata.create_all()* in cloud production, as described in **POSTGRES_MIGRATION.md**.

2. Swiggy Builders Club & OAuth 2.1 Protocol

BiteWise's key integration is its connection to the Swiggy platform via the **Swiggy Builders Club** developer program. This enables our AI Agent to place genuine queries and orders under rigorous test environments. Authentication is designed to match commercial security standards using OAuth 2.1 with PKCE.

OAuth 2.1 with PKCE Flow

To protect client secrets and prevent authorization code interception, the system implements PKCE (Proof Key for Code Exchange). When a user connects their Swiggy account, the backend generates a cryptographically secure **code_verifier** and corresponding **code_challenge**. The verifier is locked inside an HTTP-only, secure, same-site cookie, while the challenge is sent to Swiggy. Upon redirect, the code is exchanged securely backend-side.

■ Cryptographic Token Storage: AES-256-GCM

Once retrieved, Swiggy access tokens (which last 5 days) are never logged or stored in plain text. Using the Python cryptography library, the backend generates a random 12-byte Initialization Vector (IV) per encryption operation. The plaintext token is encrypted using AES-256 in Galois/Counter Mode (GCM). The resulting ciphertext, IV, and 16-byte authentication tag are packaged and saved. During decryption, the system checks the tag to guarantee that token credentials have not been modified, preventing database injection or hijacking.

Model Context Protocol (MCP) Integration

BiteWise interfaces with Swiggy utilizing the **Model Context Protocol (MCP)**. Instead of forcing the LLM to make direct REST calls, Swiggy exposes structured tools (e.g., `search_menu`, `get_addresses`, `update_food_cart`) directly into the agent's context. The agent natively invokes these tools, allowing it to browse menus, select dishes, and query active orders in a structured, standard JSON interface.

3. Robust Order Safety State Machine

Autonomous LLM agents are prone to hallucinating inputs, ordering incorrect items, or looping. To enforce rigid consumer protection, BiteWise funnels all checkout flows through an explicit **Order Safety State Machine** managed on the backend. This acts as a circuit breaker, making it impossible for the agent to place unverified orders.

Core Safety Guarantees

- **Mandatory Address Verification:** The agent is blocked from executing food searches until an explicit address ID is queried and selected from the user's saved list. This prevents ordering to wrong locations.
- **Explicit Human-in-the-Loop Confirmation:** Transitioning to the order placement state is blocked until the user clicks a manual confirmation showing the final cart items, prices, and address.
- **Strict Budget Guardrails:** A backend-enforced check automatically fails the ordering session if the total cart price exceeds **₹1,000**. This protects users from agent runaway spending.
- **COD Payment Restriction:** In compliance with Swiggy Builders sandbox parameters, the payment method is hardcoded to Cash On Delivery (COD), completely preventing actual credit/UPI billing during testing.

Order Session State Transitions

Current State	Action/Trigger	Next State
START	User selects delivery address	ADDRESS_SELECTED
ADDRESS_SELECTED	Agent starts menu search	SEARCHING
SEARCHING	Agent proposes food items	RECOMMENDATIONS_READY
RECOMMENDATIONS_READY	Agent updates cart items	CART_UPDATED
CART_UPDATED	System validates budget & limits	CART_REVIEW_READY
CART_REVIEW_READY	User clicks 'Place Order'	USER_CONFIRMED
USER_CONFIRMED	Backend places order to Swiggy	ORDER_PLACED

4. Resilience Layer & Idempotency Recovery Policy

One of the most complex challenges in digital commerce is handling **ambiguous network failures** during order submissions. If a request to place an order experiences a network timeout or a 5xx response from the server, the client cannot determine if the order failed or was processed. To solve this, BiteWise implements a custom **resilience layer** (implemented in *agent/resilience.py*) that prevents duplicate orders and charges.

The Double-Placement Prevention Protocol

The ordering flow follows a strict verification-first execution structure:

- 1. Accidental Double-Submit Pre-Check:** Prior to sending the order placement call, the resilience layer queries Swiggy's active orders (via *get_food_orders*). If a recent active order (placed within the last 60 seconds) is found in the user's records, the execution is blocked immediately and a 409 status code is returned, stopping duplicate submissions.
- 2. Ambiguous vs. Non-Ambiguous Failure Classification:** If the placement execution fails, the system categorizes the error. Explicit 4xx errors (e.g. invalid arguments, unauthenticated tokens) fail-fast closed immediately. Only network timeouts, connection terminations, or 5xx server codes are classified as *ambiguous failures*.
- 3. Status Verification Poll Loop:** Instead of retrying order placement on ambiguous failures, the system enters a polling verification state. It pauses and queries *get_food_orders* up to three times with increasing delays. If it finds a recent order matching the cart details, it maps the order ID, updates the state to successful, and recovers the session. If all three checks fail to find an active order, it marks the transaction as failed.

■ Resilience Performance Insight:

This double-pass verification guarantees that even under severe packet loss, your application acts defensively. The user is never double-charged, and order execution remains robust and deterministic regardless of API transport instability.

5. SmartPantry AI & Recipe Recommendation Engine

BiteWise features a rule-based **SmartPantry AI** system (built in *backend/pantry* and *backend/household/intelligence.py*) that tracks home food inventory and recommends recipes based on what is currently available, avoiding manual entry friction and preventing ingredient wastage.

Qualitative Stock Tracking & Automatic Cook Decay

Pantry items are categorized qualitatively as **full**, **half**, **low**, or **empty**. When a user selects a recipe and triggers the */pantry/cook/{recipe_name}* endpoint, the system decrements the stock levels of the constituent ingredients. To handle non-perishable bulk goods (e.g., rice, oil, spices), BiteWise tracks a **bulk_use_count**; bulk items only decrement in stock level every 3rd cooking cycle.

Recipe Matcher & Dietary/Allergen Filters

The recipe matching engine compares pantry stock against a template matrix of 30+ staple Indian dishes. It filters suggestions dynamically based on two layers of household constraints:

- **Dietary Preferences:** If any member of the household is registered as vegetarian/vegan, all non-veg recipes (e.g., Butter Chicken) are automatically filtered out from shared suggestion feeds.
- **Allergen Exclusions:** The system scans ingredients for registered allergies (such as eggs or peanuts) and immediately skips conflict recipes (e.g., Egg Bhurji or Poha).
- **Expiring Item Boost:** The engine computes the effective expiry date for perishable categories. Recipes utilizing ingredients expiring within 2 days are given priority status, sorting them to the top of the feed.

6. Instamart Cart Sync & Smart Grocery Planner

To close the loop between recipe planning and grocery purchasing, BiteWise integrates a **Smart Grocery Planner** that interfaces with Swiggy Instamart, allowing seamless restocking.

Recipe Shopping List Cross-Referencing

When a user schedules a recipe plan, the system checks the current pantry stock. For any ingredients that are out of stock (level 'empty') or low stock, the planner automatically generates unpurchased items in the household's active **Grocery List**. Staggered items that are already fully stocked are skipped, preventing double buying.

Fuzzy Category Classification & Stale Priority Scoring

Items added to the grocery list are parsed using a fuzzy keyword classification matching matrix to sort them into distinct store categories (Dairy, Proteins, Vegetables, Staples, Spices, Bakery). Additionally, the planner assigns a priority score to each item in the shopping list:

- **Low-Stock Urgency (+3 points):** If the item is marked as 'low' or 'empty' in the pantry, it is immediately flagged as **urgent**.
- **Staleness Penalty (+1 point):** Items that remain unpurchased on the grocery list for more than 2 days are boosted in priority to prevent shopping neglect.

Swiggy Instamart Cart Preview

The `/grocery-list/cart-preview` endpoint builds a simulated Instamart checkout cart by mapping unpurchased grocery items to real-world store listings (e.g. 'milk' is resolved to 'Nandini Fresh Milk 1L' with live pricing). This generates a structured preview including total item counts, stock statuses, and estimated total cost in Rupees.

7. Multi-User Memory Isolation & Telemetry Middleware

To support production multi-tenancy, BiteWise implements a strict security boundary at the database layer and records detailed performance telemetry per request.

SQLAlchemy Scoped Memory Isolation

User metabolic profiles (age, height, weight, activity levels) and culinary metrics (allergies, budget caps) are stored in the relational database. All queries dynamically filter against the authenticated user's session identifier (*user_id*). This scopes the SQLAlchemy transaction to the individual, ensuring data privacy.

User Profile Table Fields (SQLAlchemy Schema)

Field Name	Data Type	Purpose
user_id	VARCHAR (Primary Key)	Unique identifier scoped to authentication provider.
age / gender	INTEGER / VARCHAR	Used to calculate Basal Metabolic Rate (BMR).
height_cm / weight_kg	FLOAT / FLOAT	Used to calculate target body index and energy needs.
activity_level	VARCHAR	Multipliers applied to active daily calorie goals.
meal_budget_default	INTEGER (Default 300)	Ensures the coach filters food within user's budget.
allergies / dislikes	JSON	Enforced as strict exclusion keywords during menu filtering.

Asynchronous Request Logging & Correlation Tracing

To easily diagnose failures across our async endpoints, the FastAPI server mounts an HTTP middleware (configured in *backend/main.py*). For every incoming request, the middleware generates a unique **UUID v4**. It binds this UUID to the thread-local request state, appends it as an **X-Request-ID** custom header to the HTTP response, and logs the execution duration (in milliseconds) upon completion. This enables correlation of client actions directly to server logs during API telemetry audits.

8. DevOps, Android Hosting & PM2 Strategy

BiteWise's deployment architecture is planned to handle local debugging, high-performance containerized cloud serving (via Render/Docker), and a unique **Mini Production Server on Android** via Termux.

Multi-Stage Docker Builds

Both backend and frontend are fully dockerized. The frontend Next.js container utilizes a multi-stage compilation build pattern: stage 1 installs build dependencies, compiles TypeScript, and runs tree-shaking; stage 2 copies **ONLY** the compiled bundle into a slim Alpine node image, reducing final image sizes by over 75% for quick deployment.

Android PM2 Local Host (Mini Home Server)

For a developer portfolio, BiteWise can be run directly on an Android device running Termux, acting as a mini-server on the home Wi-Fi network. By setting up Node.js, Python, and **PM2** (Process Manager 2), we run BiteWise persistently:

```
# Start Backend FastAPI Server
pm2 start "python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000" --name "bitewise-backend"

# Start Frontend Next.js Server
pm2 start "npm run start" --name "bitewise-frontend" --cwd /path/to/frontend

# Monitor Status
pm2 status
```

Conclusion

By structuring BiteWise with a clean FastAPI + Next.js decoupled stack, establishing OAuth 2.1 with PKCE, locking transactions inside a safety state machine, and integrating Swiggy's Model Context Protocol (MCP), the application represents a production-grade integration of AI agents with secure real-world operations.